

2. Template

```
template<class T>
T add(const T& a, const T& b)
{
    return a + b;
}
```

3. Semi Infinite Data

```
random_device rd;
default_random_engine dre{ rd() };
uniform_int_distribution<int> uid{ 0, 9999 };
```

```
int max = numeric_limits<int>::min();
```

//structured binding - syntactic sugar(문법적 설탕) -> pair형으로 나와야 할 변수를 각각의 변수로 나눠서 받을 수 있게 해주는 문법

```
auto [minVal, maxVal] = minmax_element(num.begin(), num.end());
cout << "최솟값 : " << *minVal << '\n';
cout << "최댓값 : " << *maxVal << '\n';
```

4. File

```
random_device rd;
default_random_engine dre{ rd() };
uniform_int_distribution<int> uid{ numeric_limits<int>::min(), numeric_limits<int>::max() };
uniform_int_distribution<int> uidNum{ 1, 9999 };
uniform_int_distribution<int> uidNameLen{ 10, 30 };
uniform_int_distribution<int> uidChar{ 'a', 'z' };
```

```
ofstream out{ "int천만개.txt" };
ifstream in{ "int천만개.txt" };
```

```
//위의 코드처럼 txt파일로 저장하는 경우 파일 내용이 엄청 커진다.
//따라서 바이너리 모드로 파일을 기록하고 읽는 편이 빠르고 작아진다.
//이 메모리 bit 전체를 그대로 파일에 기록하는 방식이 binary mode이다.
//이번엔 1억개를 저장해 본다.
ofstream out{ "int천개.bin", ios::binary };
```

```
array<int, 1000> a;
iota(a.begin(), a.end(), 1); //a배열에 1부터 1000까지의 값을 채워넣는다.
```

```
for (int num : a)
    print("{:8}", num);
cout << endl;
```

```
//reinterpret_cast는 포인터 변환
out.write(reinterpret_cast<char*>(&a[0]), sizeof(int) * a.size());
```

```

ifstream in{ "int천개.bin", ios::binary };
if (not in)
{
    cout << "파일을 열 수 없습니다." << endl;
    system("pause");
    return(20260317);
}

```

```

in.read((char*)(a.data()), sizeof(int) * a.size());
for (int num : a)
    cout << num << '\t';
cout << endl;

```

5. Dynamic Memory Allocation

//동적 할당 -> 런타임시에 결정, 프리스토어에 할당 -> 주소로만 할당
 //정적 할당 -> 컴파일시에 결정, 스택에 할당 -> 이름 사용 가능
 //스마트 포인터를 통해 할당과 해제를 자동으로 할 수 있다.

```

//Callable Type
//C++11에서 람다식 처음 도입
//이름없는 함수 람다 - 가장 간단한 형태
//왜 이름이 없을까? - 부를 필요가 없기 때문.
//호출 가능한 타입
//1. 함수
//2. lambda
//3. 함수호출연산자를 오버로딩한 클래스 - 이 클래스로 만든 객체는 호출 가능하다.

```

```

class Dog
{
public:
    Dog() { cout << "Dog 생성" << endl; }
    ~Dog() { cout << "Dog 소멸" << endl; }
};

class smartPtr
{
public:
    smartPtr(Dog* p) : p{ p } {
        cout << "smartPtr 생성 - 자원 p를 관리 시작" << endl;
    }
    ~smartPtr() {
        delete p; //자원 해제
        cout << "smartPtr 소멸 - 자원 p를 관리 종료. 관리하던 p도 삭제" << endl;
    }
private :
    Dog* p;
};

```

```

default_random_engine dre{};
uniform_int_distribution<int> uid{ 0, 999'9999 };
array<int, 1000'0000> a;

```

```

unique_ptr<int[]> ptr;    //RAII
try {
    ptr.reset(new int[size]);    // new로 메모리를 할당받음. unique_ptr이 알아서 해제해줌.
}
catch (exception& e)
{
    cout << e.what() << endl;
    exit(404);
}

```

```

iota(ptr.get(), ptr.get() + size, 1);    // 1부터 시작하는 정수로 채우기

```

```

accumulate(ptr.get(), ptr.get() + size, 0LL);

```

//RAII로 자원관리의 어려움을 해결할 수 있다.

// 자원의 생명주기와 자원을 관리하는 객체 p의 생명주기를 일치시키면 가능하다.

// 이게 가능한 이유는 C++언어에서 지역이 끝날때 지역 객체가 반드시 소멸됨을 보장한다.

```

transform(istreambuf_iterator<char>{in}, {}, ostreambuf_iterator<char>{out},
    [](char c) {
        return toupper(c);
    }
);

```

// 걸린 시간 측정

```

//      //시작
//      auto start = chrono::high_resolution_clock::now();
//      qsort(a.data(), a.size(), sizeof(int), compare);
//      auto stop = chrono::high_resolution_clock::now();
//      //auto duration = stop - start;
//      auto duration = chrono::duration_cast<chrono::milliseconds>(stop - start);
//      cout << "qsort 걸린 시간: " << duration << endl;

```

//container

//[String]

//string은 15글자까지는 스택에 저장되고 그 이상은 프리스토어에 저장되어 포인터로 프리스토어의 주소를 가리킨다.

//unique_ptr은 포인터에 저장된 주소를 독점해야하는데. 복사하게 되면 독점이 풀리게 되므로 복제가 금지되어 있다. 그래서 복사 생성자와 복사 대입 연산자가 삭제되어 있다. -> 복사 생성자와 복사 대입 연산자가 삭제된 클래스는 복사가 금지된 클래스라고도 불린다. -> Singleton은 생성자를 삭제하는 놈으로 사용하지 않는 편이 좋다.

```

//string s{"2026년"};
//->string t = move(s);
//->string t = static_cast<string&&>(s);
//->string t{static_cast<string&&>(s)};

```

//xvalue : 이미 사라진 value

//expired value, 즉, xvalue에서 뭔가를 가져오는 건 undefined behavior이다.

//컨테이너란?

//Containers are objects that store other objects. - 컨테이너는 다른 객체를 저장하는 객체이다.

// 오브젝트란? 메모리에 올려진 클래스의 인스턴스

//STL의 컨테이너는 크게 3가지 종류로 나눈다.

//1. Sequence containers : vector, list, deque(덱), array, forward_list

//2. Associative containers : set, map -> 해싱을 위해 만들어진 구조. 키값을 기준으로 자동으로 정렬함.

//3. Unordered associative containers : unordered_set, unordered_multiset, unordered_map, unordered_multimap -> 2번보다 더 빨리 해싱할 수 있는 구조.

//Homogeneous(동질적인, 균질한) -> C++은 같은 자료형을 저장한다.

//Polymorphic(다형성) -> 컨테이너는 객체의 타입을 몰라도 저장할 수 있다. (템플릿)

//array의 입력 타입으로 element type Must be MoveConstructible and MoveAssignable. 라고 써있지만 MoveConstructible가 안되면 복사 생성자와 복사 대입 연산자가 사용된다.

//모든 컨테이너는 iterator(반복자)를 제공한다. -> 컨테이너의 요소에 접근할 수 있는 방법을 제공한다.

//lexicographically(사전식으로)

//array만 유일하게 스택에 데이터를 저장한다.

//vector는 동적 배열

//vector 24바이트에 저장하는 정보는

//첫번째 8바이트에는 현재 데이터가 실제로 저장되어 있는 데이터 개수

//두번째 8바이트에는 vector가 현재 할당한 메모리의 크기

//세번째 8바이트에는 vector가 가리키는 프리스토어 공간의 주소

//vector가 이미 할당되어 있는 메모리의 크기를 넘어서는 데이터를 저장해야할 경우,

//아예 새로운 공간을 할당받아서 그곳에 데이터를 복사, 혹은 이동해서 저장해서 포인터 주소가 가리키게 만든다.

//물론 이제 쓸모없어진 공간은 해제한다.

//이 행위의 복잡도는 $O(1)$ 인데 amortized(평균?) $O(1)$ 이라 부르며 시스템에서 메모리 할당하는 시간이 일정하지 않기때문에 그렇다.

//array만 데이터를 스택에 저장. 그 외엔 프리스토어에 저장.

//list는 .data()함수가 없다?

//data()는 Contiguous하게 데이터를 저장하는 컨테이너에서만 사용할 수 있다.